

VIT-AP UNIVERSITY

Vellore Institute of Technology (VIT)-AP, Amaravati

Department of Computer Science and Engineering

BRUTE-FORCE ATTACK PREVENTION

Submitted by:

Lokesh Chavali

Roll No: 22BCE20411

Email: chavalilokesh7@gmail.com

Submitted to:

Prof. D Santhadevi

Department of Computer Science and Engineering

VIT-AP

December 2024

Brute Force Attack Prevention System in Kali Linux (Step-by-Step Guide)

Brute force attacks involve systematically trying multiple username-password combinations until the correct credentials are found. To prevent such attacks, we implement security measures like account lockout policies, intrusion detection, and firewall rules. Below is a step-by-step process to set up a Brute Force Attack Prevention System in Kali Linux.

Step 1: Install and Configure Fail2Ban

Fail2Ban is a popular intrusion prevention tool that monitors log files for suspicious activities and blocks malicious IP addresses.

1.1 Install Fail2Ban

First, update your system and install Fail2Ban using:

`sudo apt update && sudo apt install fail2ban -y`

```
(prathap@kali)-[~]
└─$ sudo apt update && sudo apt install fail2ban -y
[sudo] password for prathap:
Hit:1 http://http.kali.org/kali kali-rolling InRelease
434 packages can be upgraded. Run 'apt list --upgradable' to see them.
fail2ban is already the newest version (1.1.0-7).
The following packages were automatically installed and are no longer required:
firebird3.0-common libconfig9v5 libgeo3.13.0 libglvnd-dev libmbedtls7v64 libsuperlu6 libx265-209 python3.12 ruby3.1-dev
firebird3.0-common-doc libconfig9 libgl1-mesa-dev libgtksourceview-3.0-1 libmsgraph-0-1 libtag1v5 openjdk-23-jre python3.12-dev ruby3.1-doc
libffi1 libdirectfb-1.7-7t64 libglapi-mesa libgtksourceview-3.0-common libpaper1 libtag1v5-vanilla openjdk-23-jre-headless python3.12-minimal
libc++1-19 libegl-dev libgles-dev libgtksourceviewmm-3.0-0v5 libpython3.12-dev libtag0 python3-appdirs python3.12-venv
libc++abi1-19 libflac12t64 libgles1 libgumbo2 libpython3.12-dev libtag0 python3-appdirs python3.12-venv
libcapstone4 libfat9 libglvnd-core-dev libjxl0.9 libqt5sensors5 libumwind-19 python3-ntlm-auth python3-setproctitle ruby-zeitwerk
libcapstone4 libfat9 libglvnd-core-dev libjxl0.9 libqt5sensors5 libumwind-19 python3-ntlm-auth python3-setproctitle ruby-zeitwerk
Use 'sudo apt autoremove' to remove them.
Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 434
```

1.2 Configure Fail2Ban for SSH Protection

1. Create a local configuration file:

`sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local`

```
(prathap@kali)-[~]
└─$ sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
```

2. Open the configuration file using a text editor:

`sudo nano /etc/fail2ban/jail.local`

```
(prathap@kali)-[~]
└─$ sudo nano /etc/fail2ban/jail.local
```

3. Find the [sshd] section and configure these settings:

[sshd]

enabled =

true port =

ssh filter =

sshd

logpath = /var/log/auth.log

maxretry = 3

bantime = 600

findtime = 300

```
[sshd]
# To use more aggressive sshd modes set filter parameter "mode" in jail.local:
# normal (default), ddos, extra or aggressive (combines all).
# See "tests/files/logs/sshd" or "filter.d/sshd.conf" for usage example and details.
#mode = normal
enabled = true
port = ssh
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 600
findtime = 300
```

- o maxretry → Number of failed attempts before a ban.
- o bantime → Duration (in seconds) for which the attacker's IP will be blocked.
- o findtime → Time period within which failed attempts are counted.

Save the file (Ctrl + X → Y → Enter).

1.3 Restart and Enable Fail2Ban

Run the following commands to apply changes:

sudo systemctl restart fail2ban

sudo systemctl enable fail2ban

```
(prathap@kali)-[~]
$ sudo systemctl restart fail2ban

(prathap@kali)-[~]
$ sudo systemctl enable fail2ban
Synchronizing state of fail2ban.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable fail2ban
```


2.2 Allow Necessary Ports

Permit SSH access and other essential services:

`sudo ufw allow 22/tcp`

```
(prathap@kali)-[~]  
$ sudo ufw allow 22/tcp  
Rule updated  
Rule updated (v6)
```

`sudo ufw allow 80/tcp`

```
(prathap@kali)-[~]  
$ sudo ufw allow 80/tcp  
Skipping adding existing rule  
Skipping adding existing rule (v6)
```

`sudo ufw allow 443/tcp`

```
(prathap@kali)-[~]  
$ sudo ufw allow 443/tcp  
Skipping adding existing rule  
Skipping adding existing rule (v6)
```

2.3 Limit SSH Connections

`sudo ufw limit ssh`

```
(prathap@kali)-[~]  
$ sudo ufw limit ssh  
Rule updated  
Rule updated (v6)
```

This allows SSH connections but restricts repeated attempts from the same IP.

2.4 Enable the Firewall

`sudo ufw enable`

```
(prathap@kali)-[~]  
$ sudo ufw enable  
Firewall is active and enabled on system startup
```

2.5 Check Firewall Status

sudo ufw status

```
(prathap@kali)-[~]
$ sudo ufw status
Status: active

To Action From
--
22/tcp LIMIT Anywhere
80/tcp ALLOW Anywhere
443/tcp ALLOW Anywhere
22/tcp (v6) LIMIT Anywhere (v6)
80/tcp (v6) ALLOW Anywhere (v6)
443/tcp (v6) ALLOW Anywhere (v6)
```

Step 3: Use TCP Wrappers for Additional Security

TCP Wrappers allow you to control access to network services.

3.1 Restrict SSH Access

Edit the hosts.allow file:

sudo nano /etc/hosts.allow

```
(prathap@kali)-[~]
$ sudo nano /etc/hosts.allow
```

Add:

sshd: 192.168.1.100

```
#
sshd: 192.168.232.135
```

Edit the hosts.deny file:

sudo nano /etc/hosts.deny

```
(prathap@kali)-[~]
$ sudo nano /etc/hosts.deny
```

Add:

sshd:

ALL


```
sshd: ALL
```

This blocks all SSH connections except the allowed ones.

Step 4: Monitor and Analyze Logs

To track failed login attempts, check the logs:

```
sudo cat /var/log/auth.log | grep "Failed password"
```

```
(prathap@kali)-[~]  
$ sudo cat /var/log/auth.log | grep "Failed password"
```

To monitor active SSH connections:

who

```
netstat -tnpa | grep 'ESTABLISHED.*sshd'
```

```
(prathap@kali)-[~]  
$ who  
  
(prathap@kali)-[~]  
$ netstat -tnpa | grep 'ESTABLISHED.*sshd'  
(Not all processes could be identified, non-owned process info  
will not be shown, you would have to be root to see it all.)
```

Step 5: Set Up Two-Factor Authentication (2FA) for Extra Security

1. Install Google Authenticator:

```
sudo apt install libpam-google-authenticator -y
```

```
(prathap@kali)-[~]  
$ sudo apt install libpam-google-authenticator -y  
libpam-google-authenticator is already the newest version (20191231-2.1).  
The following packages were automatically installed and are no longer required:  
firebird3.0-common libconfig9v5 libgeos3.13.0 libglvnd-dev libmbedcrypto7t64 libsuperlu6 libx265-209 python3.12 ruby3.1-dev  
firebird3.0-common-doc libconfig9 libgl1-mesa-dev libgtksourceview-3.0-1 libmsgpack-0-1 libtag1v5 openjdk-23-jre python3.12-dev ruby3.1-doc  
libbfiol libdirectfb-1.7-7t64 libglapi-mesa libgtksourceview-3.0-common libpaper1 libtag1v5-vanilla openjdk-23-jre-headless python3.12-minimal  
libc++abi-19 libegl-dev libgles-dev libgtksourceviewmm-3.0-0v5 libpython3.12-dev libtagc0 python3-appdirs python3.12-venv  
libc++abi1-19 libflac12t64 libgles1 libgumbo2 libqt5sensors5 libunwind-19 python3-ntlm-auth ruby-zeitwerk  
libcapstone4 libfmt9 libglvnd-core-dev libjxl0.9 libqt5webkit5 libwebrtc-audio-processing1 python3-setproctitle ruby3.1  
Use 'sudo apt autoremove' to remove them.  
  
Summary:  
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 434
```

2. Run the configuration:

google-authenticator

```
(prathap@kali)~$ google-authenticator
Do you want authentication tokens to be time-based (y/n) y
Warning: pasting the following URL into your browser exposes the OTP secret to Google:
https://www.google.com/chart?chs=200x200&chld=M|0&cht=qr&chl=otpauth://totp/prathap@kali%3Fsecret%3DXWWRDHGKT04RCRQ75U22J6GJCQ%26issuer%3Dkali



Your new secret key is: XWWRDHGKT04RCRQ75U22J6GJCQ
```

```
Enter code from app (-1 to skip): 414754
Code confirmed
Your emergency scratch codes are:
59277284
29810822
88639835
58354247
51391786
```

Do you want me to update your `"/home/prathap/.google_authenticator"` file? (y/n) y

Do you want to disallow multiple uses of the same authentication token? This restricts you to one login about every 30s, but it increases your chances to notice or even prevent man-in-the-middle attacks (y/n) y

By default, a new token is generated every 30 seconds by the mobile app. In order to compensate for possible time-skew between the client and the server, we allow an extra token before and after the current time. This allows for a time skew of up to 30 seconds between authentication server and client. If you experience problems with poor time synchronization, you can increase the window from its default size of 3 permitted codes (one previous code, the current code, the next code) to 17 permitted codes (the 8 previous codes, the current code, and the 8 next codes). This will permit for a time skew of up to 4 minutes between client and server.

Do you want to do so? (y/n) y

If the computer that you are logging into isn't hardened against brute-force login attempts, you can enable rate-limiting for the authentication module. By default, this limits attackers to no more than 3 login attempts every 30s.

Do you want to enable rate-limiting? (y/n) y

- Answer yes to enable 2FA.
- Scan the QR code with an authentication app.

3. Edit the PAM configuration file:

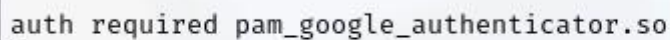
```
sudo nano /etc/pam.d/sshd
```



```
(prathap@kali)-[~]  
$ sudo nano /etc/pam.d/sshd
```

Add:

```
auth required pam_google_authenticator.so
```



```
auth required pam_google_authenticator.so  
█
```

4. Enable 2FA in the SSH config:

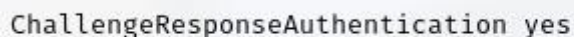
```
sudo nano /etc/ssh/sshd_config
```



```
(prathap@kali)-[~]  
$ sudo nano /etc/ssh/sshd_config
```

Find and set:

```
ChallengeResponseAuthentication yes
```



```
ChallengeResponseAuthentication yes  
█
```

5. Restart SSH service:

```
sudo systemctl restart ssh
```



```
(prathap@kali)-[~]  
$ sudo systemctl restart ssh
```

Final Testing

- Try logging in with incorrect credentials multiple times and check if Fail2Ban blocks the IP.
- Check if SSH access is restricted based on TCP Wrapper settings.
- Ensure UFW is limiting connections.

- Test 2FA authentication.

Final Steps: Verify Brute Force Protection

1 Test Incorrect Logins

Try entering the wrong password multiple times and check if Fail2Ban blocks the IP:

ssh your-username@your-server-ip

- Enter an incorrect password 3-5 times.
- Check if SSH access is blocked.

```
(prathap@kali)-[~]
$ ssh prathap@192.168.232.135
prathap@192.168.232.135's password:
Permission denied, please try again.
prathap@192.168.232.135's password:
Permission denied, please try again.
prathap@192.168.232.135's password:
prathap@192.168.232.135: Permission denied (publickey,password).
```

2 Check Fail2Ban Status

sudo fail2ban-client status sshd

- This should show banned IPs if Fail2Ban is working.

```
(prathap@kali)-[~]
$ sudo fail2ban-client status sshd
Status for the jail: sshd
├─ Filter
│ └─ Currently failed: 0
│ └─ Total failed: 2
│ └─ Journal matches: _SYSTEMD_UNIT=ssh.service + _COMM=sshd
└─ Actions
    ├─ Currently banned: 0
    ├─ Total banned: 0
    └─ Banned IP list:
```

3 Unban Yourself (If Blocked)

If you get locked out, unban your IP using:

sudo fail2ban-client set sshd unbanip 192.168.232.135

```
(prathap@kali)-[~]
$ sudo fail2ban-client set sshd unbanip 192.168.1.100
0
```

Conclusion

By following these steps, you have successfully implemented a Brute Force Attack Prevention System in Kali Linux using Fail2Ban, UFW, TCP Wrappers, and 2FA. This significantly strengthens security against brute force attacks.